Pimpri Chinchwad Education Trust's

Pimpri Chinchwad College of Engineering

Sector No. 26, Pradhikaran, Nigdi Pune -411044.

Permanently Affiliated to SPPU
Approved by AICTE, New Delhi, Govt. of Maharashtra, DTE, Mumbai.

Name: Rutuja D. Jagtap **PRN No:**123B1B146 **Class:** C1
Sub: DevOps **Academic year:** 2026-27

Assignment - 5

1. Aim

To create and run a multi-container application using Docker Compose by connecting a Flask web application with a PostgreSQL database. The application demonstrates service configuration, port mapping, environment variables, persistent storage, and container networking through a compose.yaml file.

2. Objectives

- Create a simple Flask web application that stores and displays messages.
- Define the web and database services in a compose.yaml file.
- Configure ports and database connection details using environment variables.
- Use a named volume to persist PostgreSQL data.
- Run, inspect, and test the application using Docker Compose commands.

3. Learning Outcomes

- Understand how multiple containers work together as services.
- Configure service dependencies, ports, environment variables, volumes, and networks.
- Build and start a multi-container application with Docker Compose.
- Verify database connectivity and data persistence.

4. Tools and Prerequisites

Tool / Technology	Purpose
VS Code	Create and edit the project files.
Docker Desktop	Run Docker Engine and containers.
Docker Compose	Define and manage the multi-container application.
Python / Flask	Build the web application.
PostgreSQL	Store application messages.
PowerShell / VS Code Terminal	Run Compose commands and verify services.

5. Application Overview

The browser sends requests to the Flask web service on port 5000. Flask connects to PostgreSQL on the internal Compose network. PostgreSQL data is stored in a named volume so it remains available when containers are recreated.

6. Project Structure

Create the following files inside a folder named DEVOPS_ASSIGN_5:

```
DEVOPS_ASSIGN_5/
├── app.py
├── requirements.txt
├── Dockerfile
└── compose.yaml
```

7. Flask Application — app.py

```
import os
import time
import psycopg2
from flask import Flask, request, redirect, render_template_string

app = Flask(__name__)

def connect_db():
 for _ in range(15):
 try:
 return psycopg2.connect(
 host=os.getenv("DB_HOST", "db"),
 dbname=os.getenv("POSTGRES_DB", "appdb"),
 user=os.getenv("POSTGRES_USER", "appuser"),
 password=os.getenv("POSTGRES_PASSWORD", "apppass")  )
 except psycopg2.OperationalError:
 time.sleep(2)
 raise RuntimeError("Database connection failed")

def init_db():
 with connect_db() as conn:
 with conn.cursor() as cur:
 cur.execute("""CREATE TABLE IF NOT EXISTS messages (
 id SERIAL PRIMARY KEY,
 message TEXT NOT NULL
 )""")

@app.route("/", methods=["GET", "POST"])
def home():
 init_db()
 if request.method == "POST":
 msg = request.form.get("message", "").strip()
 if msg:
 with connect_db() as conn:
 with conn.cursor() as cur:
 cur.execute("INSERT INTO messages (message) VALUES (%s)", (msg,))  return
redirect("/")
 with connect_db() as conn:
 with conn.cursor() as cur:
 cur.execute("SELECT id, message FROM messages ORDER BY id DESC")  rows =
cur.fetchall()
 return render_template_string(
 "<h2>Multi-Container Message App</h2>"
 "<form method='post'><input name='message' required>"
 "<button>Save</button></form><ul>"
 "{% for id, msg in rows %}<li>{{ id }}: {{ msg }}</li>{% endfor %}</ul>",
rows=rows)

if __name__ == "__main__":
 app.run(host="0.0.0.0", port=5000)
```

8. Dependencies — requirements.txt

```
Flask\npsycopg2-binary
```

9. Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["python", "app.py"]
```

10. Docker Compose Configuration — compose.yaml

```
services:
 web:
 build: .
 ports:
 - "5000:5000"
 environment:
 DB_HOST: db
 POSTGRES_DB: ${POSTGRES_DB:-appdb}
 POSTGRES_USER: ${POSTGRES_USER:-appuser}
 POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-apppass}
 depends_on:
 db:
 condition: service_healthy
 networks:
 - app-network

 db:
 image: postgres:16
 environment:
 POSTGRES_DB: ${POSTGRES_DB:-appdb}
 POSTGRES_USER: ${POSTGRES_USER:-appuser}
 POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-apppass}
 volumes:
 - postgres_data:/var/lib/postgresql/data
 healthcheck:
 test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER:-appuser} -d ${POSTGRES_DB:-appdb}"]
 interval: 5s
 timeout: 5s
 retries: 10
 networks:
 - app-network

volumes:
 postgres_data:

networks:
 app-network:
 driver: bridge
```

11. Configuration Explanation

Compose Feature	Configuration / Purpose
Services	web (Flask) and db (PostgreSQL) run in separate containers.
Ports	Host port 5000 maps to web container port 5000.
Environment variables	Database name, user, password, and host configure the DB connection.
Volume	postgres_data stores database files persistently.
Network	app-network lets web reach PostgreSQL using hostname db.
Dependency / healthcheck	The web service waits for PostgreSQL to become healthy.

For a real deployment, store credentials in a protected .env file or secret manager and do not commit real passwords to Git.

12. Build and Run the Application

Open the DEVOPS_ASSIGN_5 folder in VS Code. Ensure Docker Desktop is running, then run these commands in the terminal:

```
docker --version
docker compose version
docker compose up --build -d
```

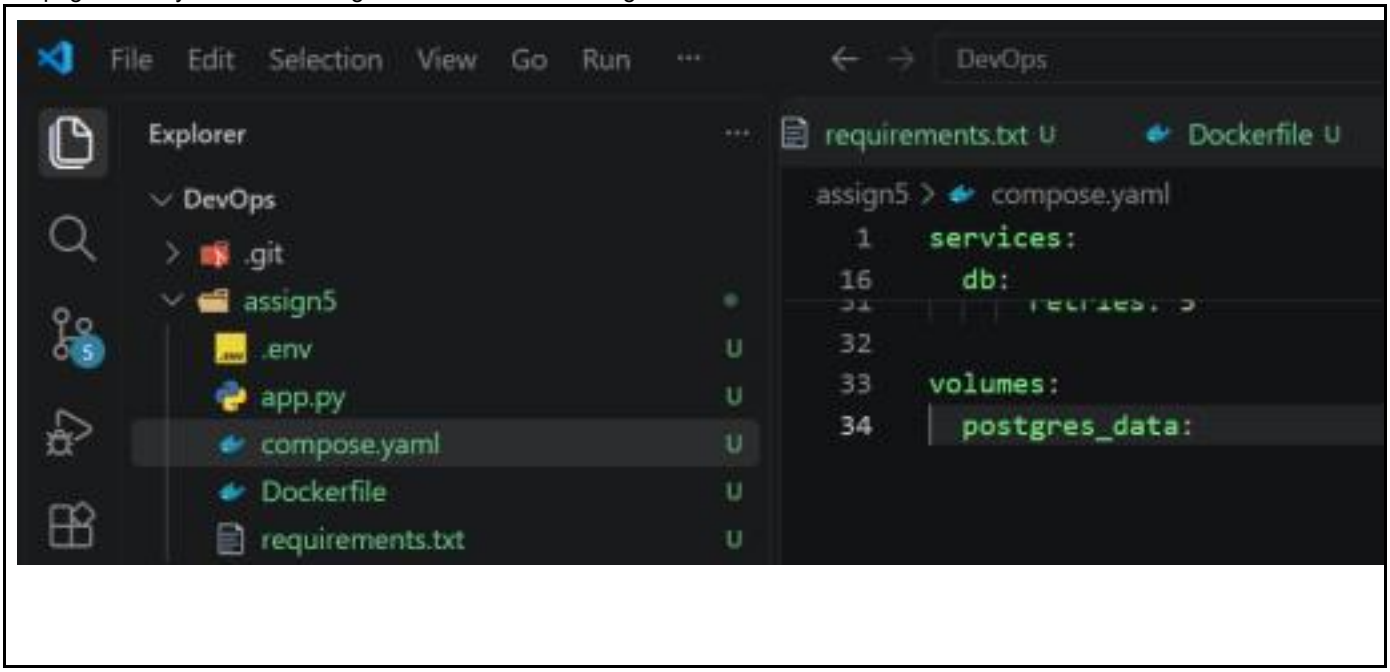
Docker Compose builds the Flask image, pulls the PostgreSQL image, creates the network and named volume, and starts both services.

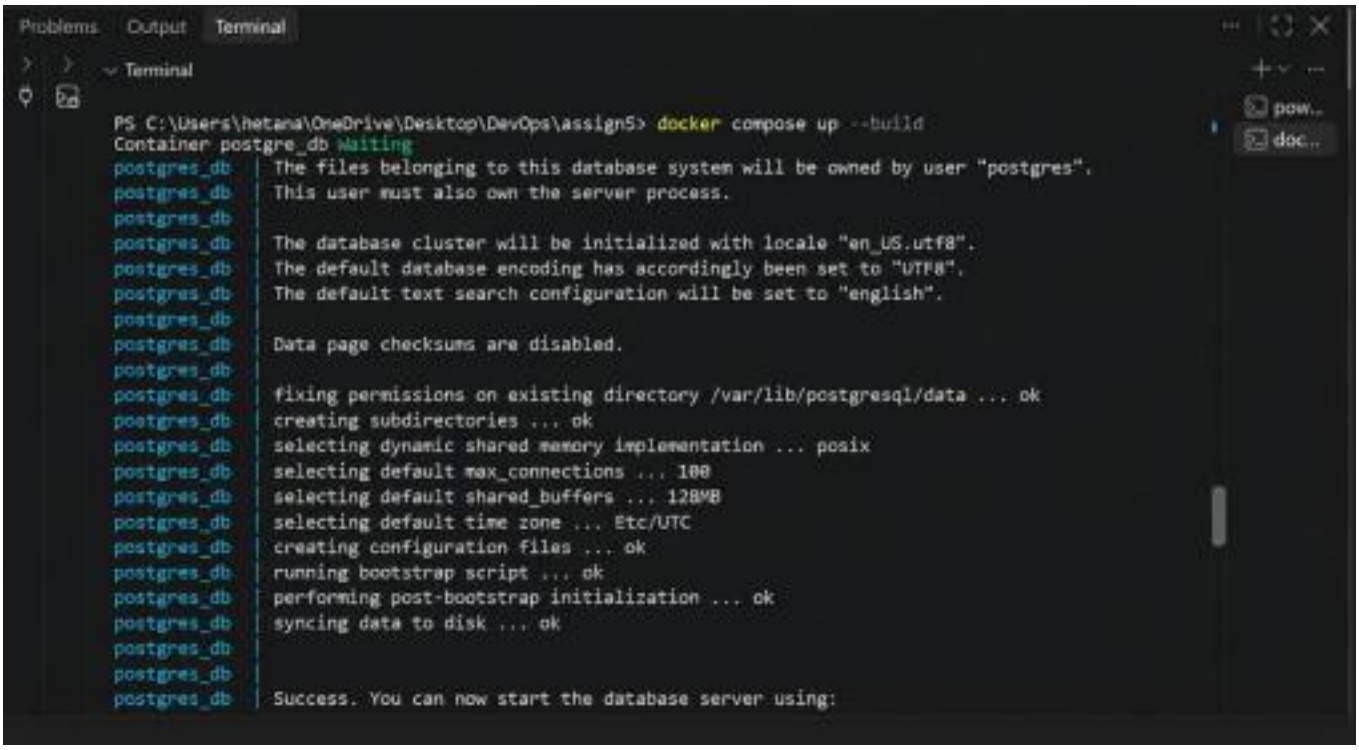
13. Verify Containers and Services

```
docker compose ps
docker compose logs
docker compose logs web
docker compose logs db
```

Both services should be running; the database should show a healthy status. The web service is published on host port 5000. **14. Test the Application**

Open the browser and visit http://localhost:5000. Enter a message and click Save. The message should appear in the list. Refresh the page to verify that the message remains stored in PostgreSQL.





15. Verify Database Persistence

Stop and restart the application containers without deleting the named volume. The previously saved message should still be visible.

```
docker compose down
docker compose up -d
docker compose ps
```

The command docker compose down removes the containers and network but keeps the named volume by default. Do not use the -v option when you want to preserve the database.

16. Useful Management Commands

Command	Purpose
docker compose up --build -d	Build and start services in the background.
docker compose ps	Show service/container status.
docker compose logs -f	Follow service logs.
docker compose exec db psql -U appuser -d appdb	Open PostgreSQL shell.
docker compose down	Stop and remove containers and network.
docker compose down -v	Also delete the database volume (data loss).

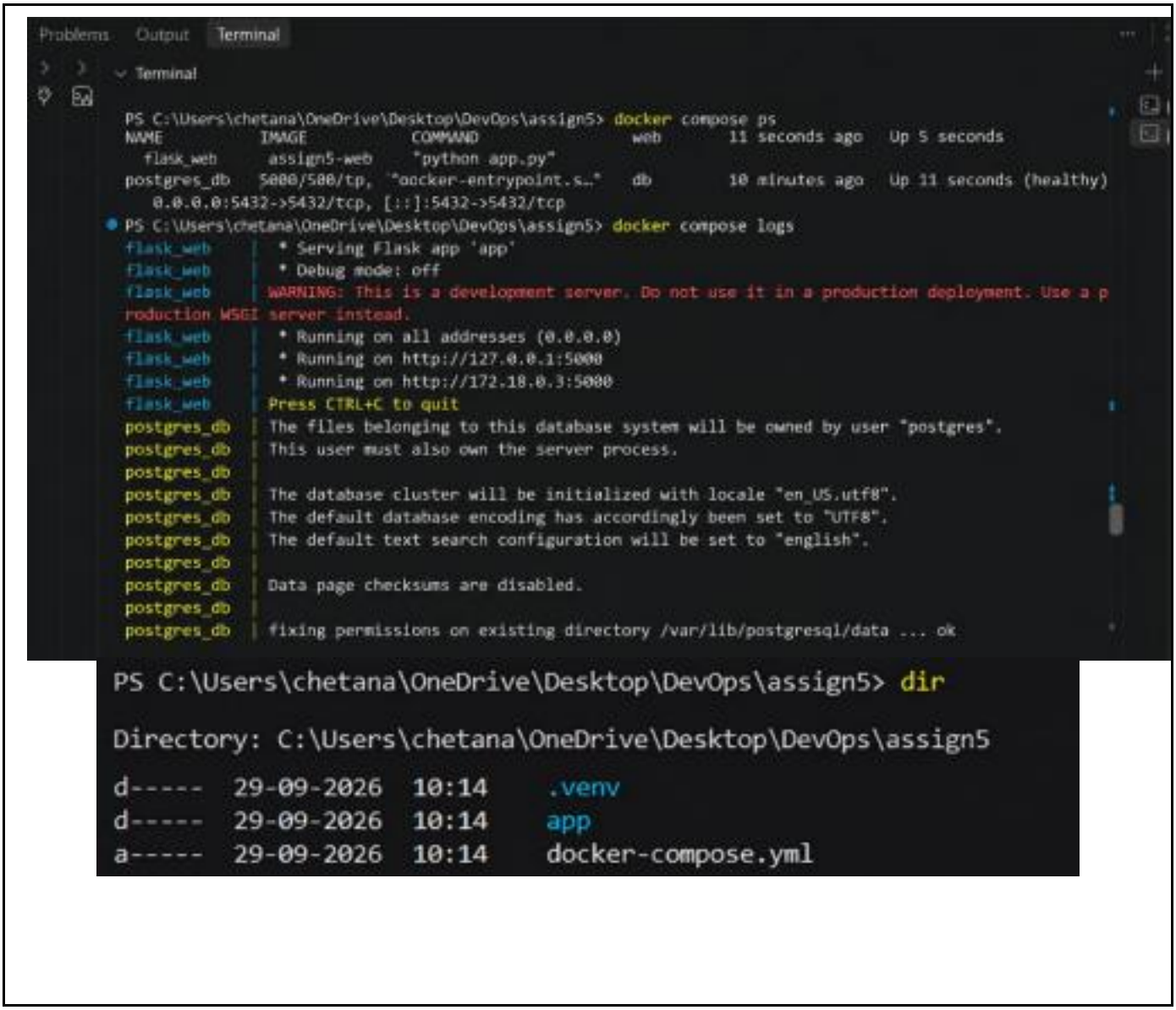
17. Multi-Container Architecture

```
Browser
  |
  | http://localhost:5000
  v
```

```
Host Port 5000 --> web container (Flask)
 |
                          | app-network / hostname: db
                         v
 db container (PostgreSQL)
 |
                         v
 postgres_data volume
```




```
PS C:\Users\chetana\OneDrive\Desktop\DevOps\assign5> docker compose up --build
[+] Building 3.2s (8/8) FINISHED                                                docker:default
 => [internal] load build definition from Dockerfile                                    0.0s
 => => transferring dockerfile: 342B                                                    0.0s
 => [internal] load metadata for docker.io/library/postgres:latest                      1.8s
 => [internal] load .dockerignore                                                       0.0s
 => => transferring context: 2B                                                         0.0s
 => [1/3] FROM docker.io/library/postgres:latest                                        1.2s
 => [2/3] COPY init.sql /docker-entrypoint-initdb.d/                                    0.1s
 => [3/3] RUN chmod +x /docker-entrypoint-initdb.d/init.sql                             0.1s
 => exporting to image                                                                  0.0s
[+] Running 2/2
 ✓ Network assign5_default  Created                                                     0.1s
 ✓ Container postgres_db    Created                                                     0.4s
```



18. Result

The Flask web application and PostgreSQL database were configured as separate services and orchestrated using Docker Compose. Port mapping enabled browser access, environment variables configured the database connection, a named volume provided persistent storage, and a dedicated network enabled communication between services.

19. Conclusion

This assignment demonstrates how Docker Compose simplifies the setup and management of a multi-container application. The web and database services can be started, monitored, and stopped together while database data is retained in a named volume.

20. Evidence to Include Before Submission

- VS Code project structure showing app.py, requirements.txt, Dockerfile, and compose.yaml.
- Terminal output of docker compose up --build -d and docker compose ps.
- Browser showing the application and a saved message.
- Logs or database output confirming successful service communication.
- Persistence check showing that the saved message remains after restarting containers.

Note: Replace the screenshot placeholders with your own execution screenshots. Do not submit placeholders as proof of execution.